KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

장바구니 미니 프로젝트

들여쓰기, items(),
모듈과 import

```
from product import Product


class Cart:
    def show(self):
        for p, q in self.items.items():
            print(p.name, q)


if __name__ == "__main__":
    cart.show()
```

오늘의 순서

1	왜 필요한가	자주 막히는 세 곳, 프로젝트 안의 위치	4–5쪽
2	들여쓰기는 상자	상자 안의 상자, for 안과 밖, return, 오류, Tab	6–11쪽
3	for p, q in items()	튜플 나눠 담기, 한 쌍씩, 긴 방법, 합계, 오류	12–18쪽
4	모듈과 import	나누는 이유, import 모양 다섯 가지, 오류	19–27쪽
5	if __name__	섞여 나오는 테스트 출력, __name__, 고치기	28–30쪽
6	실습	main.py에서 주문하기, 쿠폰 모듈 더하기	31–32쪽

3쪽은 프로젝트 준비, 33–34쪽은 막힐 때와 한 장 요약이에요.

들어가기

수업에서 만든 shopping-cart 폴더에서 해요

- a, b, c = 튜플로 꺼내 담기 13
- 딕셔너리 items(), for k, v 15
- if와 4칸 들여쓰기 17
- def, return 19
- class, __init__, 메서드, __str__ 24·25
- 클래스 변수 Product.next_id 26

shopping-cart/

product.py	상품 틀 Product
cart.py	장바구니 틀 Cart
main.py	uv init이 만든 파일
pyproject.toml	uv 프로젝트 설정

터미널

```
$ uv run cart.py
```

VS Code로 폴더를 열고 터미널에서 쳐요.

1. 왜 필요한가

장바구니 코드에서 자주 막히는 세 곳이에요

cart.py

```
1  from product import Product
2
3  class Cart:
4      def __init__(self):
5          # 빈 장바구니 생성
6          self.items = {} # {상품: 수량}
   ⋮
9      def add(self, product, qty):
10         # 품절 상품은 담을 수 없다.
11         if product.is_sold_out():
12             print("품절된 상품입니다.")
13             return
   ⋮
59     def show(self):
   ⋮
66         for p, q in self.items.items():
67             # 제품명 x 수량 = 총합계
68             print(f"{p.name:<10} x {q:<3} = {p.price * q:>12,}원")
```

3 **이 줄은 왜 맨 위에 있을까요?**
4파트 import

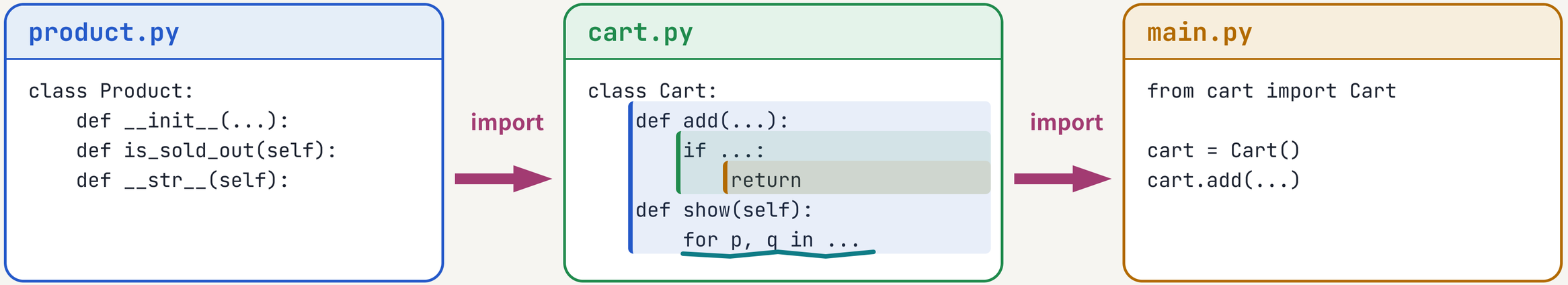
1 **이 return은 어느 줄 소속일까요?**
2파트 들여쓰기

2 **p와 q에는 무엇이 들어갈까요?**
3파트 items()

파트마다 한 곳씩 풀어요.

1. 왜 필요한가

세 가지는 모두 장바구니 프로젝트 안에 있어요



2파트 들여쓰기	파일 안에서 줄이 어느 상자 소속인지
3파트 items()	장바구니 딕셔너리를 한 쌍씩 꺼내기
4·5파트 import	파일과 파일을 잇기, 테스트 코드 막기

2. 들여쓰기는 상자

4칸 들어간 줄은 바로 위 상자 안에 있어요

cart.py

```
3  class Cart:
4      def __init__(self):                          # class Cart 안
5          # 빈 장바구니 생성                        # __init__ 안
6          self.items = {} # {상품: 수량}
7
8      # 장바구니에 상품 담기
9      def add(self, product, qty):
10         # 품절 상품은 담을 수 없다.               # add 안
11         if product.is_sold_out():
12             print("품절된 상품입니다.")             # if 안
13             return
```

1 콜론(:) 다음 줄은
4칸 더 들어가요

2 같은 칸에서 시작하는 줄은
같은 상자예요

3 왼쪽으로 나오면
그 상자가 끝나요

파이썬은 중괄호 대신 칸 수로 상자를 나눠요.

2. 들여쓰기는 상자

print(line) 한 줄의 칸 수만 다른데 출력이 달라요

for 밖에 둔 print(line)

cart.py show()

```
66  for p, q in self.items.items():
67      # 제품명 x 수량 = 총합계
68      print(f"{p.name:<10} x {q:<3} = {p.price * q:>12,}원")
69
70  print(line)
```

터미널

```
$ uv run cart.py
텀블러 1개를 담았습니다.
무선 이어폰 2개를 담았습니다.
----------------------------------------
텀블러         x 1   =       15,000원
무선 이어폰     x 2   =      178,000원
----------------------------------------
상품 합계               193,000원
배송비                        0원
결제 금액               193,000원
----------------------------------------
```

for 안에 넣은 print(line)

cart.py show()

```
66  for p, q in self.items.items():
67      # 제품명 x 수량 = 총합계
68      print(f"{p.name:<10} x {q:<3} = {p.price * q:>12,}원")
69      print(line)
```

터미널

```
$ uv run cart.py
텀블러 1개를 담았습니다.
무선 이어폰 2개를 담았습니다.
----------------------------------------
텀블러         x 1   =       15,000원
----------------------------------------
무선 이어폰     x 2   =      178,000원
----------------------------------------
상품 합계               193,000원
배송비                        0원
결제 금액               193,000원
----------------------------------------
```

← 상품마다 한 줄씩

2. 들여쓰기는 상자

return이 if 밖으로 나오면 아무것도 담기지 않아요

cart.py add()

```
 9  def add(self, product, qty):
10      # 품절 상품은 담을 수 없다.
11      if product.is_sold_out():   ← 텀블러는 품절이 아니라 False
12          print("품절된 상품입니다.")
13      return   ← if 밖이라 언제나 실행돼요
14
15      # 이미 장바구니에 담겨있는 수량     ← 여기까지 못 와요
16      in_cart_qty = self.items.get(product, 0)
17      # 더 담을 수 있는 수량
18      extra_qty = product.stock - in_cart_qty
19      if qty > extra_qty: # 재고 수량 범위를 벗어난 경우
20          print(f"재고가 부족합니다. (담을 수 있는 수량:{extra_qty}개)")
21          return
22
23      self.items[product] = self.items.get(product, 0) + qty
24      print(f"{product.name} {qty}개를 담았습니다.")
```

터미널

```
$ uv run cart.py
----------------------------------------
장바구니 비어있습니다.
----------------------------------------
상품 합계                          0원
배송비                             0원
결제 금액                          0원
----------------------------------------
```

if 안으로 4칸 더 (· 하나가 빈칸 하나)

✕ ~~····return~~

✓ ····return

2. 들여쓰기는 상자

콜론 뒤를 한 줄에 붙인 if도 같은 상자예요

cart.py 51번 줄

```
if self.is_empty():
    return 0
```

← 콜론 뒤 한 줄을 붙여 썼을 뿐이에요

← 두 줄로 풀면 4칸 들어간 if 상자예요

cart.py 52번 줄

```
return 0 if self.subtotal() >= 30_000 else 3_000
```

이건 17의 한 줄 if else예요. 둘 중 값 하나를 골라 돌려줘요

상품 합계 (uv run cart.py)	배송비
15,000원	3,000원
193,000원	0원

수업 예제는 두 줄로 적어요. 붙여 쓴 코드를 만나면 두 줄로 풀어 읽어요.

2. 들여쓰기는 상자

칸이 안 맞으면 IndentationError가 나요

터미널

```
$ uv run cart.py
  File "/home/kim/shopping-cart/cart.py", line 43
    return len(self.items) == 0
    ^^^^^^
IndentationError: expected an indented block after function definition on line 42
```

42번 줄 def 다음 줄이 안으로 안 들어갔어요

43번 줄 앞에 4칸 더 넣어요

터미널

```
$ uv run cart.py
  File "/home/kim/shopping-cart/cart.py", line 39
    print(f"{product.name} {qty}개를 뺐습니다.")
                                              ^
IndentationError: unindent does not match any outer indentation level
```

39번 줄만 10칸이라 맞는 상자가 없어요

위 줄과 같은 12칸으로 맞춰요

오류 줄에 적힌 줄 번호부터 가서 위 줄과 칸을 비교해요.

2. 들여쓰기는 상자

VS Code에서는 Tab과 Shift+Tab으로 칸을 맞춰요

`Tab`

고른 줄을 4칸 안으로

`Shift` + `Tab`

고른 줄을 4칸 밖으로

`Spaces: 4`

VS Code 맨 아래 막대 오른쪽에 보여요. Tab 한 번이 빈칸 4개예요.

cart.py

```
 9      def add(self, product, qty):
10          # 품절 상품은 담을 수 없다.
11          if product.is_sold_out():
12              print("품절된 상품입니다.")
13              return
```

12, 13번 줄을 고르고 Shift+Tab 한 번

cart.py

```
 9      def add(self, product, qty):
10          # 품절 상품은 담을 수 없다.
11          if product.is_sold_out():
12          print("품절된 상품입니다.")
13          return
```

두 줄이 if 상자 밖으로 나왔어요

스페이스를 하나씩 세지 말고 Tab과 Shift+Tab을 써요.

3. for p, q in items()

a, b = (1, 2)는 두 값을 두 칸에 나눠 담아요

play.py

```python
a, b = (1, 2)
print(a)
print(b)

name, qty = ("텀블러", 1)
print(name, qty)
```

터미널

```
$ uv run play.py
1
2
텀블러 1
```



13에서 배운 꺼내 담기예요. 왼쪽 이름 수와 오른쪽 값 수가 같아야 해요.

3. for p, q in items()

items()는 키와 값을 튜플 한 쌍씩 묶어 줘요

play.py

```
1  cart_items = {"텀블러": 1, "무선 이어폰": 2}
2
3  print(cart_items.items())
4  print(list(cart_items.items()))
```

터미널

```
$ uv run play.py
dict_items([('텀블러', 1), ('무선 이어폰', 2)])
[('텀블러', 1), ('무선 이어폰', 2)]
```

키	값
"텀블러"	1
"무선 이어폰"	2

.items()



('텀블러', 1)

('무선 이어폰', 2)

한 쌍이 튜플이니, 앞 장의 a, b = 튜플처럼 두 칸에 나눠 담을 수 있어요.

3. for p, q in items()

for가 한 쌍씩 꺼내서 name과 qty에 나눠 담아요

play.py

```
1  cart_items = {"텀블러": 1, "무선 이어폰": 2}
2
3  for name, qty in cart_items.items():
4      print(name, qty)
```

터미널

```
$ uv run play.py
텀블러 1
무선 이어폰 2
```

('텀블러', 1)

('무선 이어폰', 2)

name

"무선 이어폰"

qty

2

쌍이 두 개라 두 번 돌아요

3. for p, q in items()

for 옆에 이름 두 개는 pair[0], pair[1]을 줄인 모양이에요

긴 방법

```
for name, qty in cart_items.items():
    print(name, qty)
```

한 쌍을 pair 하나로 받고, 번호로 꺼내요

for에서 바로 두 칸에 나눠 담아요. 수업 코드는 이 모양이에요

출력이 똑같아요. pair[0]이 name, pair[1]이 qty예요.

긴 방법

터미널

```
$ uv run play.py
텀블러 1
무선 이어폰 2
```

짧은 방법

터미널

```
$ uv run play.py
텀블러 1
무선 이어폰 2
```

3. for p, q in items()

cart.items는 칸 이름, .items()는 딕셔너리 메서드예요

play.py

```python
from product import Product
from cart import Cart

tumbler = Product("텀블러", 15_000, 10)
earphone = Product("무선 이어폰", 89_000, 3)

cart = Cart()
cart.add(tumbler, 1)
cart.add(earphone, 2)

for p, q in cart.items.items():
    print(p.name, p.price, q)
```

터미널

```
$ uv run play.py
텀블러 1개를 담았습니다.
무선 이어폰 2개를 담았습니다.
텀블러 15000 1
무선 이어폰 89000 2
```



cart.items: Cart 안의 딕셔너리 칸

.items(): 딕셔너리를 한 쌍씩 꺼내는 메서드

키가 Product 객체라서 p.name, p.price를 써요

3. for p, q in items()

합계는 for로 쌓아도, sum에 넘겨도 같아요

play.py

```python
from product import Product
from cart import Cart

tumbler = Product("텀블러", 15_000, 10)
earphone = Product("무선 이어폰", 89_000, 3)
cart = Cart()
cart.add(tumbler, 1)
cart.add(earphone, 2)

total = 0
for p, q in cart.items.items():
    total = total + p.price * q
print(total)

print(sum([p.price * q for p, q in cart.items.items()]))
print(cart.subtotal())
```

터미널

```
$ uv run play.py
텀블러 1개를 담았습니다.
무선 이어폰 2개를 담았습니다.
193000
193000
193000
```

- for로 하나씩 더하기 18
- 컴프리헨션 리스트를 sum에 23
- Cart의 subtotal() 메서드 cart.py

cart.py

```python
        return sum(p.price * q for p,q in self.items.items())
```

← [ ] 없는 모양은 제너레이터, 결과는 같아요

3. for p, q in items()

.items()를 빠뜨리면 키만 나와서 둘로 못 나눠요

터미널

```
$ uv run play.py
텀블러 1개를 담았습니다.
Traceback (most recent call last):
  File "/home/kim/shopping-cart/play.py", line 8, in <module>
    for p, q in cart.items:
        ^^^^
TypeError: cannot unpack non-iterable Product object
```

cart.items를 돌면 키인 Product 하나씩 나와요.
하나를 둘로 못 나눠요

```
for p, q in cart.items.items():
```

터미널

```
$ uv run play.py
Traceback (most recent call last):
  File "/home/kim/shopping-cart/play.py", line 3, in <module>
    for name, qty in cart_items:
        ^^^^^^^^^
ValueError: too many values to unpack (expected 2)
```

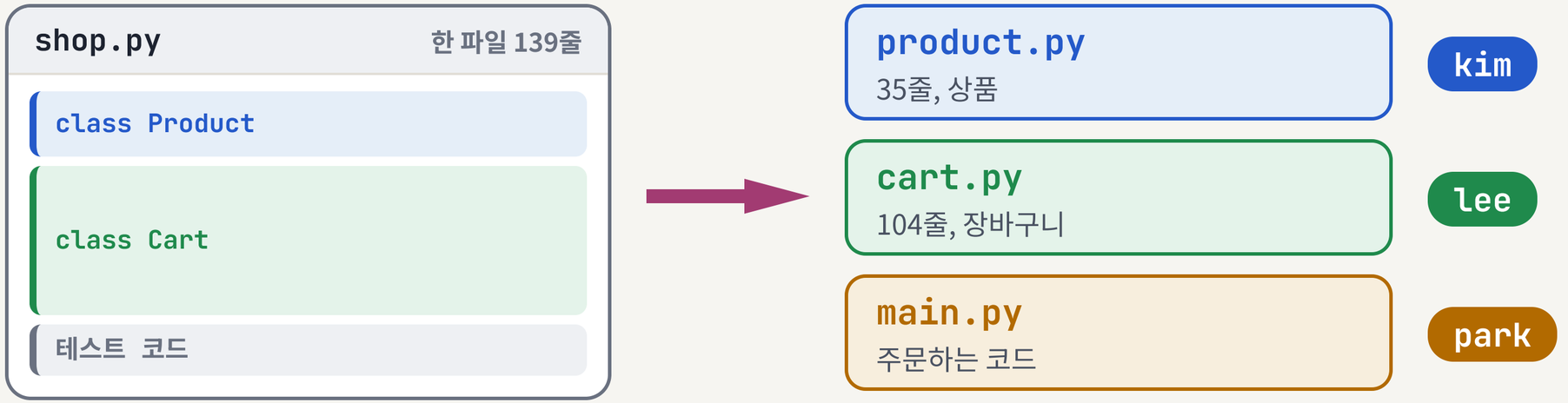
키 "텀블러"는 글자 셋으로 쪼개져서 두 칸에 안 맞아요

```
for name, qty in cart_items.items():
```

딕셔너리를 for로 돌면 키만 나와요. 키와 값이 같이 필요하면 .items()를 붙여요.

4. 모듈과 import

파일을 나누면 찾기, 같이 쓰기, 나눠 일하기가 쉬워요



1 찾기

상품을 고칠 땐 product.py만 열어요. 139줄을 뒤지지 않아요

2 같이 쓰기

Product 하나를 cart.py와 main.py가 import해서 같이 써요. 복사하지 않아요

3 나눠 일하기

06처럼 여럿이 일할 때 각자 다른 파일을 고쳐서 충돌이 줄어요

나눈 파일은 import로 다시 이어서 써요.

파일 하나가 모듈 하나예요. 이름은 .py를 뺀 부분이에요

`product.py` → `import product`

`cart.py` → `import cart`

`coupon.py` → `import coupon`

모듈 이름 짓기

영문 소문자로 지어요. 두 단어면 밑줄로 이어요.

`shop_cart.py`

띄어쓰기와 -는 import 뒤에 적을 수 없어요.

~~`shop-cart.py`~~

09에서 본 import도 같은 문법이에요

```
import fastapi
```
← uv add로 받아 온 남의 모듈

실행한 파일과 같은 폴더에 있는 모듈은 이름만 적으면 찾아요.

import product는 모듈을 통째로 불러와요

main.py

```python
import product

tumbler = product.Product("텀블러", 15_000, 10)
print(tumbler)
```

터미널

```
$ uv run main.py
1: 텀블러           15,000 재고 10개
```



꺼낼 때는 모듈 이름.안의 이름

`product.Product(...)`

어느 파일에서 온 이름인지 코드에 보여요

점(.)은 '그 안의'라는 뜻이에요. 객체의 메서드를 부를 때와 같아요.

from product import Product는 하나만 꺼내 와요

main.py

```python
from product import Product

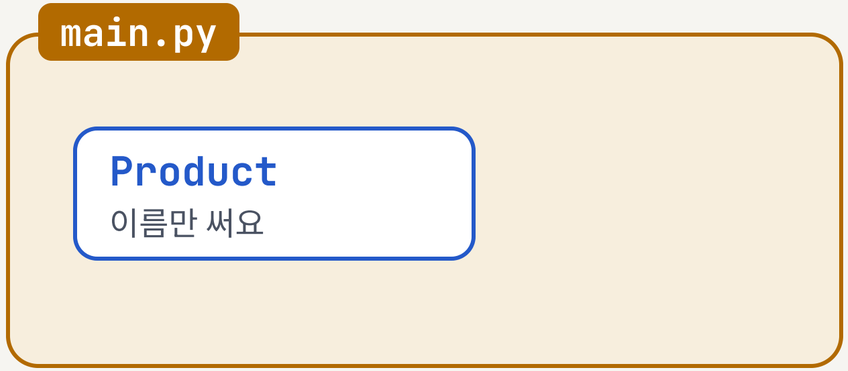
tumbler = Product("텀블러", 15_000, 10)
print(tumbler)
```

← product. 없이

터미널

```
$ uv run main.py
1: 텀블러           15,000 재고 10개
```





수업 코드 cart.py 1번 줄이 바로 이 모양이에요.

from coupon import COUPONS, discount로 여러 개를 꺼내요

coupon.py

```python
COUPONS = {"WELCOME": 3_000, "VIP": 5_000}


def discount(code):
    return COUPONS.get(code, 0)
```

main.py

```python
from coupon import COUPONS, discount

print(COUPONS)
print(discount("VIP"))
print(discount("HELLO"))
```

터미널

```
$ uv run main.py
{'WELCOME': 3000, 'VIP': 5000}
5000
0
```



쉼표로 이어서 필요한 것만 꺼내요

딕셔너리 칸도, 함수도 꺼낼 수 있어요

없는 쿠폰 HELLO는 get의 기본값 0이 나와요.

as로 모듈에 짧은 별명을 붙여요

main.py

```python
import coupon as cp

print(cp.discount("VIP"))
```

터미널

```
$ uv run main.py
5000
```

cp

coupon.py를 cp라는 이름으로

이름이 길 때 줄여 불러요

```
import pandas as pd
```

나중에 데이터 다룰 때 자주 보는 모양이에요

~~`from coupon import *`~~ ***는 모든 이름을 가져와요. 무엇이 들어왔는지 안 보여서 쓰지 않아요.**

4. 모듈과 import

모양마다 쓰는 법과 고르는 때가 달라요

모양	쓰는 법	고르는 때
`import product`	`product.Product(...)`	어느 모듈 것인지 보이게
`from product import Product`	`Product(...)`	하나를 여러 번 쓸 때
`from coupon import COUPONS, discount`	`discount("VIP")`	몇 개만 골라 쓸 때
`import coupon as cp`	`cp.discount("VIP")`	모듈 이름이 길 때
`from coupon import *`	쓰지 않아요	무엇이 왔는지 안 보여요

헷갈리면 수업 코드처럼 from 파일 import 이름부터 써요.

이름이 틀리면 ModuleNotFoundError, ImportError가 나요

터미널

```
$ uv run main.py
Traceback (most recent call last):
  File "/home/kim/shopping-cart/main.py", line 1, in <module>
    import prodcut
ModuleNotFoundError: No module named 'prodcut'
```

prodcut.py라는 파일이 없어요. 파일 이름의 오타예요 `import product`

터미널

```
$ uv run main.py
Traceback (most recent call last):
  File "/home/kim/shopping-cart/main.py", line 1, in <module>
    from product import product
ImportError: cannot import name 'product' from 'product' (/home/kim/shopping-cart/product.py). Did you mean: 'Product'?
```

product.py 안에 소문자 product가 없어요. 대소문자가 달라요 `from product import Product`

파일 이름과 대소문자를 먼저 봐요. Did you mean이 보이면 그 이름이 정답 후보예요.

4. 모듈과 import

import product만 하고 Product라고 쓰면 NameError예요

main.py

```
1  import product
2
3  tumbler = Product("텀블러", 15_000, 10)
```

터미널

```
$ uv run main.py
Traceback (most recent call last):
  File "/home/kim/shopping-cart/main.py", line 3, in <module>
    tumbler = Product("텀블러", 15_000, 10)
              ^^^^^^^
NameError: name 'Product' is not defined. Did you mean: 'product'?
```

고치기 1: 점을 찍어 써요

✓ `product.Product(...)`

고치기 2: 맨 위를 바꿔요

✓ `from product import Product`

5. if __name__ == "__main__"

cart.py를 import했더니 테스트 출력이 먼저 나와요

main.py

```python
from cart import Cart

print("main.py 시작")
cart = Cart()
```

cart.py

```python
# 제품 인스턴스 생성
tumbler = Product("텀블러", 15_000, 10)
earphone = Product("무선 이어폰", 89_000, 3)

# 카트 인스턴스 생성
cart = Cart()
cart.show()
cart.add(tumbler, 1)
cart.show()
cart.add(earphone, 2)
cart.show()
```

if 없이 파일 맨 아래에 그냥 있어요

터미널

```
$ uv run main.py
----------------------------------------
장바구니 비어있습니다.
----------------------------------------
상품 합계                        0원
배송비                          0원
결제 금액                        0원
----------------------------------------
텀블러 1개를 담았습니다.
----------------------------------------
텀블러          x 1    =        15,000원
----------------------------------------
상품 합계                   15,000원
배송비                       3,000원
결제 금액                   18,000원
----------------------------------------
무선 이어폰 2개를 담았습니다.
----------------------------------------
텀블러          x 1    =        15,000원
무선 이어폰       x 2    =      178,000원
----------------------------------------
상품 합계                  193,000원
배송비                          0원
```

main.py 시작은 맨 끝

5. if __name__ == "__main__"

직접 실행한 파일만 __name__이 "__main__"이에요

cart.py

```
⋮
90 print("cart.py의 __name__:", __name__)
```

main.py

```
1 from cart import Cart
2
3 print("main.py의 __name__:", __name__)
```

터미널

```
$ uv run cart.py
cart.py의 __name__: __main__
```

터미널

```
$ uv run main.py
cart.py의 __name__: cart
main.py의 __name__: __main__
```

cart.py가 실행된 방법	cart.py의 __name__
uv run cart.py로 직접 실행	"__main__"
다른 파일이 import로 부름	"cart" (모듈 이름)

5. if __name__ == "__main__"

테스트 코드는 if __name__ == "__main__": 안에 넣어요

cart.py

```
91  # 테스트 코드
92  if __name__ == "__main__":  ← 직접 실행할 때만 True
93      tumbler = Product("텀블러", 15_000, 10)
94      earphone = Product("무선 이어폰", 89_000, 3)
95
96      cart = Cart()
97      cart.add(tumbler, 1)
98      cart.add(earphone, 2)
99      cart.show()
```

product.py 26번 줄에는 이미 이 줄이 있어요. 그래서 product.py를 import해도 상품 목록이 찍히지 않았어요.

product.py

```
26  if __name__ == "__main__":
```

터미널

```
$ uv run main.py
main.py 시작
```

import할 때는 테스트가 안 돌아요

터미널

```
$ uv run cart.py
텀블러 1개를 담았습니다.
무선 이어폰 2개를 담았습니다.
----------------------------------------
텀블러          x 1    =        15,000원
무선 이어폰      x 2    =       178,000원
----------------------------------------
상품 합계                  193,000원
배송비                        0원
결제 금액                  193,000원
----------------------------------------
```

6. 실습

main.py에서 상품과 장바구니를 불러 주문해요

main.py

```python
from product import Product
from cart import Cart

tumbler = Product("텀블러", 15_000, 10)
earphone = Product("무선 이어폰", 89_000, 3)
pad = Product("마우스패드", 9_000, 0)

cart = Cart()
cart.add(tumbler, 2)
cart.add(earphone, 1)
cart.add(pad, 1)
cart.checkout()

print(tumbler)
print(earphone)
```

← 두 파일에서 하나씩

← 재고 0이라 품절

← 영수증을 찍어요

터미널

```
$ uv run main.py
텀블러 2개를 담았습니다.
무선 이어폰 1개를 담았습니다.
품절된 상품입니다.
----------------------------------------
텀블러          x 2    =        30,000원
무선 이어폰      x 1    =        89,000원
----------------------------------------
상품 합계                  119,000원
배송비                        0원
결제 금액                  119,000원
----------------------------------------
결제가 완료되었습니다. 감사합니다!
1: 텀블러            15,000 재고 8개
2: 무선 이어폰        89,000 재고 2개
```

텀블러 재고가 10개에서 8개로 줄었어요

상품은 product.py, 장바구니는 cart.py, 주문은 main.py가 맡아요.

6. 실습

coupon.py를 더해 세 가지를 한 번에 써요

main.py

```python
from product import Product
from cart import Cart
from coupon import COUPONS, discount

for code, amount in COUPONS.items():
    print(f"{code:<8} {amount:>6,}원 할인")

tumbler = Product("텀블러", 15_000, 10)
cart = Cart()
cart.add(tumbler, 3)

pay = cart.total() - discount("VIP")
print(f"결제할 금액: {pay:,}원")
```

coupon.py

```python
COUPONS = {"WELCOME": 3_000, "VIP": 5_000}


def discount(code):
    return COUPONS.get(code, 0)
```

터미널

```
$ uv run main.py
WELCOME   3,000원 할인
VIP       5,000원 할인
텀블러 3개를 담았습니다.
결제할 금액: 40,000원
```

45,000원 - VIP 5,000원

들여쓰기

def와 for 안은 4칸

items()

`for code, amount in COUPONS.items()`

import

`from coupon import COUPONS, discount`

정리

막히면 오류의 마지막 줄을 봐요

1	**`IndentationError: expected an indented block`** 콜론 다음 줄이 안으로 안 들어갔어요	**4칸 넣기**
2	**`IndentationError: unindent does not match ...`** 위 줄들과 칸 수가 안 맞아요	**`Tab, Shift+Tab`**
3	**`TypeError: cannot unpack non-iterable Product object`** 딕셔너리를 .items() 없이 돌았어요	**`.items()`**
4	**`ModuleNotFoundError: No module named ...`** 파일 이름 오타, 다른 폴더에서 실행	**파일 이름 확인**
5	**`ImportError: cannot import name ...`** 모듈 안에 그 이름이 없어요. 대소문자	**`Did you mean`**
6	**import했더니 테스트 출력이 나와요** 테스트 코드가 if __name__ 밖에 있어요	**`if __name__ == "__main__":`**

uv run은 프로젝트 폴더(shopping-cart) 안에서 쳐요.

한 장 요약

콜론(:) 다음 줄
4칸 더 들어가서 그 상자 안

Tab / Shift+Tab
고른 줄을 4칸 안으로, 밖으로

a, b = (1, 2)
튜플을 두 칸에 나눠 담기

for p, q in d.items():
키와 값을 한 쌍씩 꺼내기

import product
product.Product(...)로 쓰기

from product import Product
Product(...)로 바로 쓰기

from coupon import A, B
쉼표로 여러 개, as로 별명

if __name__ == "__main__":
직접 실행할 때만 테스트

파일 안은 들여쓰기로, 파일 사이는 import로 나눠요.

다음 수업에서 장바구니에 기능을 더할 때도 새 파일을 만들고 import로 이어 붙여요.